

Tugas 1 - VM Manager

Sistem Operasi

Fakultas Ilmu Komputer, Universitas Indonesia

Gasal 2026/2027

Kelas : C

Kelompok : C4

Varian : E

Nama	NPM
Khanyfatul Muflikhat	2506589755
Syakira Aqilah Amru	2506541622
Naurah Claradinda Aulia Pane	2506657163
Husainah Syamsiah	2506589036

[TAUTAN_VIDEO_PRESENTASI_C04](#)

Pembagian Kerja

1. Khanyfatul Muflikhat (NPM 2506589755) - Host Lead

Bertanggung jawab penuh atas skrip `vm_ctl.sh` pada bagian kontrol dasar VM, meliputi fungsi `list`, `info`, `start`, dan `stop`. Turut menangani validasi input dan penanganan error pada seluruh fungsi tersebut. Selain itu, bertanggung jawab dalam pengelolaan repository GitHub kelompok serta proses editing video presentasi.

2. Syakira Aqilah Amru (NPM 2506541622) - Host Snapshot

Bertanggung jawab atas skrip `vm_ctl.sh` pada bagian manajemen snapshot, meliputi fungsi `snapshot create` dan `snapshot list`. Selain itu, bertugas melakukan testing gabungan terhadap keseluruhan `vm_ctl.sh` serta menyusun dokumentasi command sisi host.

3. Naurah Claradinda Aulia Pane (NPM 2506657163) - Guest `sysinfo.sh`

Bertanggung jawab atas skrip `sysinfo.sh`, meliputi pengecekan informasi kernel, jumlah pengguna, jumlah proses berjalan, dan deteksi virtualisasi, serta logic pengambilan metrik disk usage (`df`) dan

jumlah vCPU (nproc) sesuai varian kelompok. Bertanggung jawab pula menghasilkan berkas laporan akhir sysinfo_report.txt.

4. Husainah Syamsiah (NPM 2506589036) - Program C

Bertanggung jawab atas program resource_check.c, meliputi pembacaan data dari stdin, logic penentuan status PASS/WARN/FAIL untuk metrik disk usage, serta logic PASS/FAIL untuk perbandingan jumlah vCPU. Bertanggung jawab pula melakukan testing integrasi pipe antara sysinfo.sh dan resource_check.c.

Penjelasan [HOST SIDE: LEAD]

Penjelasan

HOST LEAD : list, info, start, stop

Bagian ini menjelaskan implementasi vm_ctl.sh untuk fungsi kontrol dasar VM dari sisi host, yaitu list, info, start, dan stop. Script dirancang secara modular, di mana setiap aksi dibungkus dalam fungsi Bash terpisah dan dipanggil melalui sebuah case statement sebagai router utama berdasarkan argumen command line yang diberikan user.

a. Fungsi **vm_list()**

Tujuan: Menampilkan seluruh VM yang terdaftar di VirtualBox pada host.

Command inti: VBoxManage list vms

Cara kerja: Output mentah dari VBoxManage list vms berformat "nama_vm" {uuid}. Script melakukan filtering dengan sed untuk membuang bagian UUID, kemudian diberi penomoran otomatis menggunakan nl agar formatnya sesuai contoh output pada spesifikasi tugas.

Alasan desain: UUID tidak relevan untuk ditampilkan ke user karena user berinteraksi dengan nama VM, bukan ID internal VirtualBox.

b. Fungsi **vm_info()**

Tujuan: Menampilkan RAM, vCPU yang dialokasikan, serta status VM saat ini.

Command inti: VBoxManage showvminfo <nama_vm> --machinereadable

Cara kerja: Flag --machinereadable dipilih karena menghasilkan output berformat key=value per baris, yang jauh lebih mudah diparsing secara otomatis dibanding output default showvminfo yang berformat naratif untuk dibaca manusia. Script mengekstrak tiga baris spesifik menggunakan grep dan cut: memory= (RAM), cpus= (jumlah vCPU), dan VMState= (status: running/poweroff/saved).

c. Fungsi **vm_start()**

Tujuan: Menyalakan VM secara headless (tanpa membuka jendela GUI).

Command inti: VBoxManage startvm <nama_vm> --type headless

Cara kerja: Flag --type headless memastikan VM berjalan di background tanpa antarmuka grafis, sesuai requirement tugas. Setelah command dijalankan, script memeriksa exit code untuk menentukan apakah proses start berhasil atau gagal, lalu menampilkan status yang sesuai ke user.

d. Fungsi **vm_stop()**

Tujuan: Mematikan VM secara aman, bukan dipaksa.

Command inti: VBoxManage controlvm <nama_vm> acpipowerbutton

Cara kerja: Command ini mensimulasikan penekanan tombol power fisik pada VM, mengirim sinyal ACPI ke sistem operasi di dalam guest. Sistem operasi (Ubuntu) kemudian menangani proses shutdown secara normal, menutup proses dan service satu per satu.

Alasan desain : Dipilih dibanding VBoxManage controlvm <vm> poweroff, karena poweroff menyebabkan VM langsung mati tanpa proses shutdown yang semestinya dan berisiko menyebabkan corrupt data atau filesystem di dalam VM. Oleh karena itu acpipowerbutton dipilih.

PENJELASAN VALIDASI DAN ERROR HANDLING

Setiap fungsi yang membutuhkan nama VM sebagai argumen (info, start, stop) melakukan pengecekan apakah argumen tersebut kosong sebelum menjalankan command VBoxManage. Pada start dan stop, validasi ditambah dengan pemanggilan vm_exists untuk memastikan VM yang dimaksud benar-benar terdaftar sebelum perintah dieksekusi. Pada vm_info, dilakukan pengecekan tambahan apakah hasil showvminfo kosong, untuk mencegah script menampilkan output kosong/rusak jika user salah mengetik nama VM.

Selain itu, seluruh pemanggilan VBoxManage showvminfo dan VBoxManage list vms dibersihkan dari karakter carriage return (\r) menggunakan tr -d '\r', untuk mengantisipasi ketidaksesuaian hasil perbandingan string ketika script dijalankan pada environment Bash di Windows (Git Bash atau WSL).

DOKUMENTASI OUTPUT: Pemanggilan **list**, **info**, **start** dan **stop** di terminal

```
KHANYFATUL MUFLIKHAT@LAPTOP-EGATQ94D MINGW64 ~/OneDrive/Dokumen/Semester 3/OS/os-tugas-1 (featu
$ ./vm_ctl.sh list
=====
TUGAS 1 OS - KELOMPOK C4
=====
Memindai daftar Virtual Machine...

Daftar VM terdaftar:
1. Ubuntu-Robotik
2. OS262-Base-Image-amd64
3. os-tugas-1

KHANYFATUL MUFLIKHAT@LAPTOP-EGATQ94D MINGW64 ~/OneDrive/Dokumen/Semester 3/OS/os-tugas-1 (featu
$ ./vm_ctl.sh info os-tugas-1
=====
TUGAS 1 OS - KELOMPOK C4
=====
VM                : os-tugas-1
RAM dialokasikan  : 4096 MB
vCPU dialokasikan : 2
Status saat ini   : powered off

KHANYFATUL MUFLIKHAT@LAPTOP-EGATQ94D MINGW64 ~/OneDrive/Dokumen/Semester 3/OS/os-tugas-1 (featu
$ ./vm_ctl.sh start os-tugas-1
=====
TUGAS 1 OS - KELOMPOK C4
=====
Menyalakan VM 'os-tugas-1' secara headless...
Waiting for VM "os-tugas-1" to power on...
VM "os-tugas-1" has been successfully started.
VM 'os-tugas-1' berhasil dinyalakan. Status: running
```

```
KHANYFATUL MUFLIKHAT@LAPTOP-EGATQ94D MINGW64 ~/OneDrive/Dokumen/Semester 3/OS/os-tugas-1 (featur
$ ./vm_ctl.sh stop os-tugas-1
=====
TUGAS 1 OS - KELOMPOK C4
=====
Mematikan VM 'os-tugas-1' secara aman...
Perintah shutdown terkirim. Menunggu VM benar-benar mati...
VM 'os-tugas-1' berhasil dimatikan. Status: powered off

KHANYFATUL MUFLIKHAT@LAPTOP-EGATQ94D MINGW64 ~/OneDrive/Dokumen/Semester 3/OS/os-tugas-1 (featur
$ ./vm_ctl.sh info os-tugas-1
=====
TUGAS 1 OS - KELOMPOK C4
=====
VM                : os-tugas-1
RAM dialokasikan  : 4096 MB
vCPU dialokasikan : 2
Status saat ini   : powered off

KHANYFATUL MUFLIKHAT@LAPTOP-EGATQ94D MINGW64 ~/OneDrive/Dokumen/Semester 3/OS/os-tugas-1 (featur
$ ./vm_ctl.sh info vm-tidak-ada
=====
TUGAS 1 OS - KELOMPOK C4
=====
Error: VM 'vm-tidak-ada' tidak ditemukan.

KHANYFATUL MUFLIKHAT@LAPTOP-EGATQ94D MINGW64 ~/OneDrive/Dokumen/Semester 3/OS/os-tugas-1 (featur
$ ./vm_ctl.sh start
=====
TUGAS 1 OS - KELOMPOK C4
=====
Error: nama VM belum diisi. Gunakan: ./vm_ctl.sh start <nama_vm>

KHANYFATUL MUFLIKHAT@LAPTOP-EGATQ94D MINGW64 ~/OneDrive/Dokumen/Semester 3/OS/os-tugas-1 (featur
$ ./vm_ctl.sh start vm-tidak-ada
=====
TUGAS 1 OS - KELOMPOK C4
=====
Error: VM 'vm-tidak-ada' tidak ditemukan. Cek dengan: ./vm_ctl.sh list

KHANYFATUL MUFLIKHAT@LAPTOP-EGATQ94D MINGW64 ~/OneDrive/Dokumen/Semester 3/OS/os-tugas-1 (featur
$ ./vm_ctl.sh stop vm-tidak-ada
=====
TUGAS 1 OS - KELOMPOK C4
=====
Error: VM 'vm-tidak-ada' tidak ditemukan. Cek dengan: ./vm_ctl.sh list
```

Gambar 1. Output pemanggilan command list, info, start, dan stop di terminal

Penjelasan [HOST SIDE: SNAPSHOT]

Penjelasan

HOST SNAPSHOT: snapshot create, snapshot list

Melanjutkan bagian sebelumnya, bagian ini menjelaskan implementasi `vm_ctl.sh` secara spesifik untuk fitur *snapshot* dari sisi *host* yang mencakup **snapshot create** dan **snapshot list**. Adapun terdapat pula tambahan *helper function* `vm_exists()` untuk validasi eksistensi VM.

a. *Helper function* `vm_exists()`

Tujuan: memastikan bahwa VM sudah terdaftar

Command inti: `VBoxManage list vms`

Cara kerja: fungsi akan menjalankan `VBoxManage list vms`, membuang karakter *carriage return* Windows dan mengidentifikasi nama VM menggunakan `grep -q`. Hasilnya 0 jika ditemukan, 1 jika tidak. Karena merupakan fungsi pembantu, fungsi ini tidak didesain untuk mencetak apapun ke layar.

Alasan desain: validasi keberadaan VM dibutuhkan untuk menjalankan berbagai fungsi (tidak hanya *snapshot* tetapi juga *start* dan *stop*).

b. Fungsi `vm_snapshot_create()`

Tujuan: menyimpan *check-point* atau dalam hal ini disebut *snapshot* pada VM tertentu

Command inti: `VBoxManage snapshot <nama_vm> take <nama_snapshot>`

Cara kerja:

- **Validasi** ⇒ script akan mulai dengan memvalidasi terlebih dahulu apakah argumen nama vm dan nama snapshot sudah terisi, kemudian lanjut memvalidasi keberadaan VM.
- **Formatting** ⇒ waktu saat ini akan diambil menggunakan `date` kemudian diubah formatnya (spasi menjadi *underscore* dan titik dua menjadi tanda hubung) untuk digabungkan dengan nama *snapshot* sehingga format akhirnya akan tersimpan sebagai `<nama>_<YYYY-MM-DD>_<HH-MM-SS>`
- **Main task & output** ⇒ perintah `take` dijalankan untuk membuat *snapshot* dengan output bawaan VirtualBox akan dibuang ke `/dev/null` dan script akan mencetak pesan berhasil apabila *snapshot* sukses dibuat atau pesan *error* jika sebaliknya.

Alasan desain: cap waktu ditempelkan pada tahap pembuatan (*formatting*), bukan ketika menampilkan *snapshot list* sehingga nama yang muncul di VirtualBox akan konsisten. Output bawaan dari `VBoxManage` disembunyikan supaya pesan yang ditampilkan seragam dengan *command* lain.

c. Fungsi `vm_snapshot_list()`

Tujuan: menampilkan daftar *snapshot* pada VM tertentu

Command inti: `VBoxManage snapshot <nama_vm> list --machinereadable`

Cara kerja:

- **Validasi** ⇒ sama seperti `vm_snapshot_create()`, script akan mulai dengan memvalidasi argumen dan keberadaan VM.
- **Main task & output** ⇒ script mengambil daftar *snapshot* dengan format *machine readable* yang menghasilkan baris dengan pola `SnapshotName=<nama>`. *Carriage return* akan dibuang oleh `tr` dan hanya yang diawali `SnapshotName` yang disaring dengan `grep`. Jika hasilnya kosong, VM dianggap belum memiliki *snapshot* dan script akan mencetak pesan yang sesuai. Jika ada isinya, `sed` akan membuang `SnapshotName=` beserta tanda kutip dan menyisakan nama *snapshot* murni. Kemudian, loop `while` akan mencetak setiap nama *snapshot* dengan urutan angka.

Alasan desain: Penyaringan dengan `grep '^SnapshotName'` dilakukan pada tahap pengambilan data sehingga status "belum memiliki *snapshot*" tetap akurat meskipun VirtualBox menyisipkan baris lain (misalnya `CurrentSnapshotName`) pada output mentahnya.

DOKUMENTASI OUTPUT: Pemanggilan *snapshot list* dan *snapshot create* di terminal

```

syaki@Syakira MINGW64 ~/college/term_3/os/os-tugas-1 (feature/host-vm-ctl)
$ VM="os-tugas-1"

syaki@Syakira MINGW64 ~/college/term_3/os/os-tugas-1 (feature/host-vm-ctl)
$ ./vm_ctl.sh snapshot list "$VM"
=====
TUGAS 1 OS - KELOMPOK C4
=====
VM 'os-tugas-1' belum memiliki snapshot.

syaki@Syakira MINGW64 ~/college/term_3/os/os-tugas-1 (feature/host-vm-ctl)
$ ./vm_ctl.sh snapshot create "$VM" checkpoint-1
=====
TUGAS 1 OS - KELOMPOK C4
=====
Membuat snapshot 'checkpoint-1' pada VM 'os-tugas-1'...
Snapshot 'checkpoint-1' berhasil dibuat pada 2026-09-21 19:59:08.

syaki@Syakira MINGW64 ~/college/term_3/os/os-tugas-1 (feature/host-vm-ctl)
$ ./vm_ctl.sh snapshot create "$VM" checkpoint-2
=====
TUGAS 1 OS - KELOMPOK C4
=====
Membuat snapshot 'checkpoint-2' pada VM 'os-tugas-1'...
Snapshot 'checkpoint-2' berhasil dibuat pada 2026-09-21 19:59:13.

syaki@Syakira MINGW64 ~/college/term_3/os/os-tugas-1 (feature/host-vm-ctl)
$ ./vm_ctl.sh snapshot list "$VM"
=====
TUGAS 1 OS - KELOMPOK C4
=====
Daftar snapshot VM 'os-tugas-1':
  1. checkpoint-1_2026-09-21_19-59-08
  2. checkpoint-2_2026-09-21_19-59-13

syaki@Syakira MINGW64 ~/college/term_3/os/os-tugas-1 (feature/host-vm-ctl)
$ ./vm_ctl.sh snapshot list vm-not-exist
=====
TUGAS 1 OS - KELOMPOK C4
=====
Error: VM 'vm-not-exist' tidak ditemukan. Cek dengan: ./vm_ctl.sh list

```

Gambar 2. Output pemanggilan fungsi snapshot di terminal

Alur per *command*:

- Inisialisasi VM ke dalam suatu variabel untuk memudahkan penulisan *command*
- Pemanggilan fungsi **snapshot list** memicu pesan keluaran bahwa VM belum memiliki *snapshot*
- Pemanggilan fungsi **snapshot create** dengan nama **checkpoint-1** memicu pesan keluaran bahwa *snapshot* berhasil dibuat disertai informasi waktu pembuatan *snapshot* pertama
- Pemanggilan fungsi **snapshot create** dengan nama **checkpoint-2** memicu pesan keluaran bahwa *snapshot* berhasil dibuat disertai informasi waktu pembuatan *snapshot* kedua
- Pemanggilan fungsi **snapshot list** yang memicu pesan keluaran daftar *snapshot* yang tertera (dalam hal ini adalah **checkpoint-1** dan **checkpoint-2** yang digabungkan dengan waktu pembuatannya sebagai format akhir)
- **[Error Handling]**: Pemanggilan fungsi **snapshot list** dengan VM yang tidak valid (VM tidak eksis) memicu pesan keluaran *error* yang menyatakan bahwa VM tidak ditemukan

Penjelasan [GUEST SIDE: [SYSINFO.SH](#)]

Penjelasan
GUEST — sysinfo.sh: kernel, user, proses, deteksi virtualisasi, disk usage, nproc, fitur tambahan Bagian ini menjelaskan implementasi <code>sysinfo.sh</code> untuk mengecek kondisi sistem dari dalam VM (guest). Script mengumpulkan beberapa informasi dasar sistem, menghitung metrik sesuai Varian E, mengirim data ke program C untuk dievaluasi, lalu menyusun semua hasil menjadi satu laporan. a. Info OS/Kernel Tujuan: Menampilkan nama sistem operasi dan versi kernel yang sedang berjalan. Command inti: <code>uname -sr</code> Cara kerja: Flag <code>-s</code> menampilkan nama sistem (Linux), dan <code>-r</code> menampilkan versi kernel. Command ini dipilih karena langsung memberikan info yang dibutuhkan tanpa perlu parsing tambahan. b. Jumlah Akun Pengguna Biasa Tujuan: Menghitung berapa akun pengguna biasa yang terdaftar di sistem (bukan akun sistem seperti root). Command inti: <code>awk -F: '\$3 >= 1000 && \$3 < 60000 {print \$1}' /etc/passwd wc -l</code> Cara kerja: File <code>/etc/passwd</code> berisi daftar semua akun di sistem, dengan UID (User ID) sebagai kolom ketiga. Akun sistem biasanya punya UID kecil, sedangkan akun pengguna biasa punya UID mulai dari 1000. Script memfilter baris dengan UID di rentang 1000-59999, lalu menghitung jumlahnya dengan <code>wc -l</code>. c. Jumlah Proses Berjalan Tujuan: Menghitung total proses yang sedang aktif di sistem. Command inti: <code>ps aux wc -l</code> Cara kerja: <code>ps aux</code> menampilkan semua proses yang berjalan, satu proses per baris. Hasilnya dihitung dengan <code>wc -l</code>, lalu dikurangi 1 karena baris pertama adalah header tabel, bukan proses. d. Deteksi Virtualisasi Tujuan: Memastikan sistem benar-benar berjalan di dalam VM, bukan mesin fisik. Command inti: <code>sudo dmidecode -s system-product-name</code> Cara kerja: <code>dmidecode</code> membaca informasi hardware dari BIOS/firmware. Jika hasilnya mengandung kata "VirtualBox", berarti sistem ini dikonfirmasi berjalan di dalam virtual machine. e. Fitur Tambahan: Uptime VM Tujuan: Menampilkan berapa lama VM sudah menyala sejak terakhir boot. Command inti: <code>uptime -p</code> Alasan pemilihan: Fitur ini dipilih karena sederhana untuk diimplementasikan tetapi tetap berguna untuk monitoring, misalnya untuk mengetahui apakah VM sering di-restart atau sudah berjalan stabil dalam waktu lama. f. Metrik Varian E — Disk Usage Tujuan: Mengambil persentase penggunaan disk pada root filesystem. Command inti: <code>df -h / awk 'NR==2 {print \$5}' tr -d '%'</code>

Cara kerja: `df -h /` menampilkan info penggunaan disk. Baris kedua (`NR==2`) berisi data yang dibutuhkan, kolom kelima adalah persentase pemakaian. Simbol `%` dihapus agar hasilnya berupa angka murni yang bisa diproses lebih lanjut.

g. Metrik Varian E — Jumlah vCPU Guest

Tujuan: Mengambil jumlah vCPU yang terlihat dari dalam VM.

Command inti: `nproc`

Cara kerja: Command ini langsung mengembalikan jumlah core processor yang dialokasikan ke VM, untuk dibandingkan dengan alokasi dari sisi host.

h. Integrasi dengan Program C

Tujuan: Mengirim dua metrik (disk usage & `nproc`) untuk dievaluasi status PASS/WARN/FAIL-nya.

Command inti: `echo "$disk_usage $nproc_guest" | ./resource_check`

Cara kerja: Kedua angka dikirim sebagai satu baris teks yang dipisah spasi, disalurkan melalui pipe ke program `resource_check` sebagai standard input (bukan argumen command line), sesuai ketentuan tugas. Program C membaca kedua angka tersebut dan mengembalikan status evaluasi melalui standard output.

```
naurah@os262:~$ ./sysinfo.sh
=====
TUGAS 1 OS – KELOMPOK C4
=====
Mengecek sistem...
OS/Kernel       : Linux 7.0.0-30-generic
Akun pengguna   : 2 akun
Proses berjalan : 131 proses
[sudo: authenticate] Password:
Virtualisasi    : Terdeteksi (VirtualBox)
Uptime VM       : up 1 minute

Menghitung metrik varian kelompok...
Disk usage      : 20%
vCPU guest      : 2
Disk Status     : PASS
vCPU Status     : PASS

Menyimpan laporan ke sysinfo_report.txt...
Laporan berhasil disimpan.
```

```
naurah@os262:~$ cat sysinfo_report.txt
```

```
TUGAS 1 OS - KELOMPOK C4
```

Check Category	Item	Status	Details
OS	Linux 7.0.0-30-generic	PASS	Kernel info
Users	Regular accounts	PASS	2 akun
Processes	Running	PASS	131 proses
Virtualization	Hypervisor	PASS	Terdeteksi (VirtualBox)
Uptime	VM	PASS	up 1 minute
Disk	20%	PASS	Threshold check
vCPU	guest:2	PASS	vs alokasi host

Penjelasan [GUEST SIDE: RESOURCE_CHECK.C]

Penjelasan

GUEST - resource_check.c : metrik 1 pengecekan disk usage, metrik 2 perbandingan jumlah host allocation dan vCPU yang teralokasikan

Program ini mengambil input berupa 2 integer dalam format (%d %d) yang merupakan input persentase disk usage dan jumlah vCPU guest, *respectively*.

Jumlah alokasi host yang seharusnya di-define sebagai konstanta karena kecil kemungkinan untuk diubah. Selanjutnya untuk mengecek status disk usage juga di-define ambang batas untuk FAIL dan WARN yang merupakan 75 dan 90 sesuai dengan ketentuan tugas.

a. Metrik 1: Disk Usage (%)

Tujuan: Mengecek kesehatan penggunaan disk.

Program: Logika if-else yang membandingkan input disk usage dengan ambang batas yang menentukan status.

Cara kerja: Program akan membandingkan input disk usage, jika sudah mencapai 90 atau lebih maka akan memberi output FAIL, jika mencapai 75 atau lebih, namun kurang dari 90 maka akan memberi output WARN, jika kurang dari 75 maka status yang di-output adalah PASS.

b. Metrik 2 (Varian E): Jumlah vCPU guest (nproc) vs alokasi host

Tujuan: Memastikan jumlah vCPU yang dialokasikan host sama dengan jumlah vCPU yang ada pada guest.

Program: Logika if-else yang membandingkan input vCPU guest dan alokasi host yang sudah dijadikan konstanta 2 sesuai dengan jumlah yang diatur host.

Cara kerja: Program akan membandingkan input vCPU guest dan alokasi host, jika jumlahnya sama maka akan meng-output PASS. Jika beda maka output-nya FAIL.

